

WERGONIC · PRODUCT

Product Documentation

A complete feature map of the Wergonic platform — mobile app, web client panel and admin panel — including user roles, flows, route maps, and the known gaps found in the code.

DOCUMENT

Product reference

GENERATED

17 March 2026

SOURCE

Branch **dev** — all 3 repositories

Contents

1	Platform Overview	What is Wergonic · User Roles · The 3 Interfaces · Supported Languages
2	Authentication & User Management	Login Methods · Password Reset Flow · Session/Token Management · Organization Management · ...
3	Mobile App — Complete Feature Map	Onboarding & Setup · Session Recording · Lite Mode · Sensor Management · ...
4	Web Client Panel (cloud.wergonic.com) — Complete Feature Map	Dashboard (/dashboard) · Sessions (/dashboard/sessions) · Assessments (/dashboard/assessments) · Reports & AI Generation · ...
5	Admin Panel — Complete Feature Map	Routes · Organization Management · User Management · Device Management · ...
6	Notifications & Feedback Systems	In-App Notifications (Mobile) · Web Notifications · Snackbar Notifications (Web) · Error Messages · ...
7	Known Issues & Gaps (from code analysis)	Unfinished Features · Placeholder/Dummy Elements · Dead Code · Statistics & Metrics Concerns · ...
8	Complete Route Map	Web Client Panel Routes · Admin Panel Routes · Mobile App Routes

Platform Overview

What is Wergonic

Wergonic is an ergonomic risk assessment platform that combines smart workwear sensors (Movesense 9-axis IMU via Bluetooth Low Energy), a Flutter mobile app for field recording, a React web dashboard for analysis, and a Django backend for data processing and AI-powered reporting. The platform enables ergonomists and occupational health professionals to objectively measure and assess workplace ergonomic risks using standardized methods (RAMP, RULA, REBA, MEC) with real-time sensor data, rather than relying solely on manual observation.

User Roles

ROLE	DESCRIPTION	ACCESS
WERGONIC_ADMIN (Superadmin)	Platform-level administrator	Admin Panel + Client Panel + full backend access
ORG_ADMIN	Organization administrator	Client Panel — manage users, sessions, assessments for their org
ERGONOMIST	Ergonomic assessor / consultant	Client Panel — record sessions, create assessments, generate reports
WORKER	Field worker being assessed	Mobile App only — cannot access web Client Panel

The 3 Interfaces

1. **Admin Panel** (`wergonic-admin`) — Superadmin-only. Manages organizations, users across orgs, devices, firmware.
2. **Client Panel** (`client-panel` / cloud.wergonic.com) — Main user-facing web app for ORG_ADMIN and ERGONOMIST roles. Sessions, assessments, reports, work cycles, user management.
3. **Mobile App** (`wergonic-flutter` / ergotech) — Flutter app for field workers and ergonomists. Sensor pairing, calibration, session recording, offline data collection, sync.

Supported Languages

From i18n configuration across all apps:

- **English** (en) — default/fallback
- **Swedish** (sv)
- **German** (de)
- **Spanish** (es)
- **Dutch** (nl / nl-NL)

Authentication & User Management

Login Methods

Email Magic Link (Passwordless)

- User enters email on login page
- Backend sends a sign-in link via email (Firebase Auth)
- User clicks link → redirected to `/auth/process-login?link=<encoded>`
- Frontend completes Firebase sign-in, receives ID token
- Token used for all subsequent API calls
- Mobile app has its own magic link endpoint (`/users/auth/mobile-passwordless-signin/`)

Email + Password

- Standard email/password login via Firebase Auth
- Available on both web and mobile

Worker Account Creation

- Public endpoint at `/users/create-worker-account/`
- Web form at `/auth/signup` (AccountCreationPage)
- Creates a WORKER role user

Worker Blocking on Web: If a WORKER logs into the Client Panel, they are immediately signed out with error message "Workers cannot access this application." Workers can only use the mobile app.

Password Reset Flow

1. User visits `/auth/password-reset` → enters email
2. Backend sends reset email via Firebase
3. User clicks link → `/auth/password-reset/new`
4. User enters new password → backend updates Firebase + Django password
5. Confirmation email sent

Session/Token Management

- Firebase Authentication manages tokens
- ID tokens automatically refreshed by Firebase SDK
- Axios interceptor attaches `Authorization: Bearer {idToken}` to every API request
- Backend verifies Firebase token on each request, updates `last_seen` timestamp
- Custom claims on Firebase token include: `{role, organization_id}`

Organization Management

Multi-Organization Support

- Users can belong to multiple organizations via `UserOrganization` join table

- Each membership has its own role (a user can be ERGONOMIST in one org, ORG_ADMIN in another)
- One organization is "active" at a time — user can switch active org
- Switching active org: POST `/users/organizations/{orgId}/change-to-active/`

Organization Registration

- Public endpoint at `/organizations/register/`
- Creates organization + admin user in one step

Organization Fields

- Name (unique), Country, Address, Industry (A-U industry codes)
- Contact phone, Company registration number
- Logo (URL, uploaded to Firebase Storage)
- Subscription info: `current_subscription` , `license_expiration`
- Limits: `max_number_workers` (default 500), `max_number_admins` (default 5), `max_number_sessions_month` (default 3000), `max_number_assessments_month` (default 3000)
- AI engine selection: Gemini (default) or Chat-button-001
- Hardware kit access: OWN, BORROWED, NO_ACCESS

Invitation System

- Admin sends invitation to email with role assignment
- Invitation has 7-day expiration
- Accept: POST `/organizations/invitation/{invitationId}/`
- Creates UserOrganization record + Profile if needed
- Sends welcome email

User Profile

User Fields

- Email, first name, last name, role, unit
- Profile image (URL)
- Personal number (employee ID)
- Subject ID (research/study ID)
- Locale preference
- AI consent flag
- RAMP limit display preference (TIME or PERCENTAGE)

Worker Profile (Extended)

- Weight, resting heart rate
- Gender (Male, Female, Other)
- Dominant arm (Right, Left)
- Date of birth, age
- Comment field
- Consultant assignment (FK to another user — the ergonomist managing this worker)
- `is_main` flag (primary profile)

Role Permissions

ACTION	WERGONIC_ADMIN	ORG_ADMIN	ERGONOMIST	WORKER
Access Admin Panel	Yes	No	No	No
Access Client Panel	Yes	Yes	Yes	No
Access Mobile App	Yes	Yes	Yes	Yes
Manage organizations (create/archive)	Yes	No	No	No
Manage org users	Yes	Yes	No	No
Deactivate users	Yes	Yes	No	No
Create/edit sessions	Yes	Yes	Yes	No
View sessions (own org)	Yes	Yes	Yes	Own only
Create assessments	Yes	Yes	Yes	No
Generate AI reports	Yes	Yes	Yes	No
Manage devices/firmware	Yes	No	No	No
Manage work cycles	Yes	Yes	Yes	No
Manage categories/labels	Yes	Yes	Yes	No

Mobile App — Complete Feature Map

3.1 Onboarding & Setup

First Launch Experience

1. **Splash Screen** (/) — App initialization, Firebase auth check
2. If not logged in → **Login Page** (/login)
 - OAuth provider buttons
 - "Login with Email" option → /login_with_email
3. Post-login → **Welcome Screen** (/welcome)
 - Checks for firmware updates
 - Organization selection if user belongs to multiple orgs

Sensor Pairing Process

1. Navigate to **Pairing Screen** (/work_session_config/pairing)
2. App scans for Movesense sensors via MDS SDK
3. Discovered sensors listed with name/ID
4. User selects sensor → assigns to a limb (rightArm, leftArm, trunk, head, hand, forearm, chestStrap)
5. MDS establishes BLE connection → receives serial number confirmation
6. Sensor stored in local database with limb assignment
7. Multiple sensors can be paired — one per limb, up to 7 total

Calibration Process

Full Calibration (/sensor-calibration — LimbsCalibrationPage):

For each connected limb (2-4 steps depending on limb type):

Step 1 — Pre-Establish Streams:

- Opens IMU data streams for all devices
- Reduces latency during actual calibration

Step 2 — Device Validation:

- Verifies all expected devices are connected and receiving data

Step 3 — Natural Position Capture:

- User holds limb in neutral/relaxed posture
- System captures ~2 seconds of IMU data
- Computes quaternion orientation reference

Step 4 — Bent Position Capture (trunk/head only):

- User bends to maximum comfortable range
- System captures ~2 seconds
- Creates second reference point for range-of-motion

Calibration storage: CalibrationDataEntity saved per limb with quaternion + posture state + timestamp.

Per-Limb Recalibration (/recalibration — LimbsRecalibrationPage):

- Allows recalibrating a single limb without redoing all

Organization Selection

- **Switch Organization** (/settings/switch_organization)
- Lists all organizations user belongs to
- Switching changes active org context for all operations

3.2 Session Recording

Work Session Configuration (/work_session_config)

- Select session type: Work Task or Work Cycle
- Select worker profile (who is being assessed)
- Select workplace/tag
- Select which limbs to record (via SelectableLimbsCubit)
- Connect sensors → go to pairing if needed
- Calibrate → go to calibration if needed

Work Task Sessions

Full flow:

1. Configure session (worker, limbs, workplace)
2. Connect & calibrate sensors
3. Start recording → **Work Session Recording Screen** (/work_session)
4. Live metrics displayed during recording (see below)
5. User can add work tasks/breaks via **Work Tasks Screen** (/work_session/WorkTasks)
6. Pause/resume as needed
7. End session → data saved locally → sync to cloud

Work Cycle Sessions

- Pre-defined cycle from Factory → Line → Workstation → Task hierarchy
- Select cycle via **Work Cycle Screen** (/work_session_config/work-cycle-screen)
- Select specific tasks via **Work Cycle Tasks List** (/work_session_config/work-cycle-task-screen)
- Auto-advance between tasks (if `timeBasedWorkCycles` enabled, advances every `newWorkCycleFrequency` minutes, default 180)
- Same recording UI as Work Task, but with structured task progression

Data Collected During Recording

- **Per limb (BLE sensors):** Accelerometer (3-axis), Gyroscope (3-axis), computed Quaternion, Elevation Angle, Angular Velocity, Pitch, Roll, Yaw, Temperature
- **Heart rate (chestStrap):** BPM, confidence, RR intervals
- **Location (if enabled):** GPS latitude, longitude, altitude, accuracy, speed
- **Activity recognition:** Walking, running, driving, still, biking (via phone sensors)
- **Session events:** Task start/end, break start/end (user-marked)

Live Metrics During Recording

The recording screen shows real-time:

- **Elevation Angle** — current angle + yellow/red threshold indicators
- **Angular Velocity** — current speed + accumulated over intervals
- **Trunk Bending** — forward and side angles
- **Heart Rate** — BPM (if chestStrap connected)
- **Vibration Detection** — confidence percentage for hand tremor/shaking
- **Activity Mode** — walking/driving/still/running/biking icon
- **Session Timer** — elapsed recording time
- **Sensor Status** — connection state per limb (connected/disconnected/reconnecting)
- **Battery Level** — per sensor (checked every `batteryCheckIntervalSeconds` , default 10s)

Voice/Audio Feedback

- Configurable voice guidance intervals (10-60 seconds)
- Announces posture warnings when thresholds exceeded
- Controlled by `isAudioFeedbackEnabled` and `voiceFeedbackIntervals` settings
- Limb feedback priority configurable via `limbsFeedbackOrder`

Pause/Resume Behavior

Pause:

- Timer stops
- BLE subscriptions suspended
- CSV writes stop queuing
- UI shows pause state with resume button

Resume:

- Timer resumes from paused time
- Re-establishes BLE subscriptions
- CSV writes resume
- Time gap handled (no data loss, just a jump)

Session Events (Tasks & Breaks)

- During recording, user can mark task start/end and break start/end
- Each event creates a `SessionEventEntity` with:
 - Event type: taskEvent or breakEvent
 - Event name (user-entered or from work label)
 - Phase: start or finish
 - Timestamp
- Events are synced to backend and used for per-task analysis

Background Execution

- Android foreground service keeps recording when app is backgrounded
- Notification shows "Recording in progress"
- Battery optimization exemption requested (`disable_battery_optimization`)
- BLE connections maintained via MDS SDK
- **Screen lock:** Recording continues (foreground service)

- **Device reboot:** Session data saved to local DB, but active recording is lost. Session remains in local storage for later sync.

Session End

1. All CSV writers flushed (force write buffered data)
2. File handles closed
3. Session directory compressed into ZIP
4. Local database updated: `syncStatus` → `createdNotSynced`
5. Session saved to local DB with all metadata
6. **Session Summary Page** (`/session_summary`) shown with:
 - Duration
 - Connected devices summary
 - Sync status
 - Option to view report or sync immediately

3.3 Lite Mode

What is Lite Mode

Phone-based recording using the device's built-in accelerometer and gyroscope instead of external Movesense BLE sensors.

How It Differs from Full Mode

ASPECT	FULL MODE (BLE)	LITE MODE (PHONE)
Sensors	External Movesense	Phone accelerometer/gyroscope
Limbs	Up to 7 simultaneously	1 only (phone placement)
Accuracy	High (calibrated IMU)	Lower (consumer phone sensors)
Heart Rate	Via chestStrap	Manual entry only
Calibration	Multi-step per limb	Single 5-second phone calibration
Cost	Requires sensor hardware	No additional hardware

Lite Mode Calibration (`/LiteMode`)

1. User places phone on chest/torso
2. Stand still in natural position for ~5 seconds
3. System captures reference quaternion from phone sensors
4. Single-point calibration stored

Limitations

- Only 1 limb tracked at a time
- No multi-sensor reference data (hand/forearm)
- Lower accuracy from consumer-grade phone sensors

- No heart rate without manual input
- Sampling rate depends on phone hardware (typically 50-100 Hz vs consistent 104 Hz from Movesense)

3.4 Sensor Management

Sensor Discovery & Pairing

- BLE scan using MDS SDK (`MdsAsync.startScan()`)
- Scans for Movesense sensors by advertised name
- DFU mode: Separate scan using flutter_blue_plus filtering for "dfu" names
- Discovered sensors show: name, ID, signal strength

Connection Per Sensor

1. Stop scanning
2. `Mds.connect(id)` → MDS SDK handles connection
3. Callback receives serial number on success
4. BLE connection parameters set: min interval 48.75ms, max 100ms
5. Device stored in local DB with limb assignment

Multi-Sensor Support

- **Supported limbs:** rightArm, leftArm, trunk, head, hand, forearm, chestStrap (7 total)
- **Typical usage:** 3-5 sensors (both arms + trunk, optionally head + hand/forearm)
- **Limit:** One sensor per limb, up to 7 simultaneous connections
- Managed via `SensorConnectionCubit` tracking `Map<Limbs, BleDeviceEntity>`

Sensor Status Indicators

- **Connected** — green, actively receiving data
- **Disconnected** — red, no data
- **Reconnecting** — yellow/orange, attempting reconnection
- **Scanning** — blue, discovery in progress

Battery Information

- Checked periodically during sessions (default every 10 seconds)
- Tracked per device via `SensorBatteryCubit`
- Displayed as percentage

Firmware Information

- Device firmware version retrieved via MDS `/Info` path
- Available firmware listed from backend (`/devices/firmwares/`)
- **Firmware Update Screen** (`/firmware`) — DFU via Nordic DFU library
- `firmware_update_available_dialog.dart` — prompts user when update available

flag: Firmware update dialog flow appears incomplete (TODO in code)

Disconnect Behavior

- MDS callback fires on disconnect
- `SensorConnectionCubit` updates state to disconnected
- During recording: automatic reconnection attempted

Reconnection Behavior

- `reconnectMissingDevices()` called periodically during recording
- Up to 3 retries per device, 10-second intervals
- Clears Android GATT cache before retry
- Re-initiates full `Mds.connect()` flow
- Manual reconnection available from settings

3.5 Offline Mode

What Works Offline

- Full session recording (BLE + lite mode)
- Sensor pairing and calibration
- Session history viewing (locally stored sessions)
- Past reports (if previously cached)
- All settings changes
- Work label/task creation for session events

What Doesn't Work Offline

- Session sync/upload to cloud
- Work cycle download/refresh
- Work category refresh
- New user login (requires Firebase auth)
- AI report generation
- Firmware download

Sync When Back Online

- `InternetCubit` monitors connectivity state
- Sessions marked `notSynced` or `createdNotSynced` in queue
- User can manually trigger sync from session history
- No automatic retry with delay — relies on user action or background check on reconnect

Visual Indicators

- Online/offline state shown in UI
- Sessions show sync status badge:
 - `notSynced` — local only, not yet uploaded
 - `createdNotSynced` — remote session created, files not uploaded
 - `historyNotSynced` — old session needing resync
 - `fullySynced` — fully synced to cloud

flag: No automatic retry mechanism for failed syncs. User must manually trigger.

3.6 Data & History

Session List / History (/sessions_listing and /session_history)

- Lists all sessions (local + synced)
- Shows: worker name, date, duration, sync status, limbs recorded
- Filter/sort capabilities

Session Summary (/session_summary)

- Quick stats view post-session
- Duration, connected devices, sync status
- Navigation to full report

Report Viewing (/report)

- Full session analysis report
- Risk data per limb: arm angular velocity, arm elevation angle, trunk bending, head bending, HR
- Charts and visualizations
- Color-coded risk levels

TNO Report (/tnoReport)

- Task-based ergonomic risk assessment
- Joint angle analysis results
- Static vs dynamic movement breakdown
- Time spent in each angle range per limb

Data Export

- Session data exported as ZIP (CSV files per limb)
- Export happens during sync process
- No standalone export-to-phone feature visible

3.7 Settings & Configuration

All Mobile App Settings

Measurement Settings:

- `feedbackType` — Immediate vs Accumulated feedback
- `voiceFeedbackIntervals` — 10-60 seconds between voice alerts
- `feedbackRefreshRate` — 50-200 ms (controls UI feedback latency)
- `standardImuRate` — 52, 104, or 200 Hz (BLE sample rate)
- `refImuRate` — 52, 104, or 200 Hz (reference limb rate)

Sensor Settings:

- `gyroRange` — Gyroscope full-scale: ± 250 , ± 500 , ± 1000 , ± 2000 °/s

- `accRange` — Accelerometer full-scale: ± 2 , ± 4 , ± 8 , ± 16 G
- `integralGainValue` — Mahony filter integral gain (default 0.0)
- `proportionalGainValue` — Mahony filter proportional gain (default ~ 2.0)
- `headSensorPosition` — chestOriented vs trunkOriented

Session Settings:

- `workSessionType` — WorkTask vs WorkCycle
- `timeBasedWorkCycles` — Auto-advance cycles by time
- `newWorkCycleFrequency` — Minutes between auto-advance (default 180)
- `sessionDurationLimit` — Max session hours (default 8)
- `tnoArchiveGeneration` — Auto-compute TNO after session

Calibration:

- `isTwoPointCalibrationEnabled` — Toggle 2-point vs 1-point calibration

Vibration Detection:

- `isVibrationDetectionEnabled` — Detect hand tremor/shaking
- `useImprovedVibrationDetection` — Physics-based vs simple variance
- `confidenceThresholdHigh` / `confidenceThresholdLow` — Detection sensitivity

System:

- `liteMode` — Enable phone sensor mode
- `isHrActive` — Enable heart rate tracking
- `offlineMode` — Disable cloud sync
- `autoStartSession` — Auto-begin recording on app launch
- `enableLocation` — Track GPS during session
- `appLanguage` — EN, NL, SV, DE, ES
- `batteryCheckIntervalSeconds` — 5-60 seconds (default 10)
- `limbsFeedbackOrder` — Priority list for feedback

Developer Mode Features

- Performance overlay (FPS, memory, frame time)
- CSV performance benchmarking
- Sensor data logs (raw IMU output)
- Shake gesture → Report Bug Sheet
- `useNewCsvSystem` — Toggle between legacy and new CSV writer
- `showExtraTiles` — Show advanced measurement tiles
- `isPerformanceOverlayEnabled` — FPS counter

Report Problem (`/reportProblem`)

- Bug/issue reporting form
- Creates ticket via `/tickets/` endpoint
- Attachments supported

Web Client Panel (cloud.wergonic.com) — Complete Feature Map

4.1 Dashboard (/dashboard)

What's on the Dashboard

- **WelcomeCard** — Greeting with user name
- **UsersStatistics** — User count metrics for the organization
- **SessionsStatistics** — Session metrics with date range filtering
- **LicenseStatistics** — License usage and expiration info

Data Source

- All dashboard data fetched via `useRetrieveActiveOrganizationStatsQuery`
- Calls: GET `/organizations/stats/?user_start_date&user_end_date&session_start_date&session_end_date&RAMP_start_date&RAMP_end_date`
- Date range parameters allow filtering statistics by time period

Stats Available

- Total users in org
- Total sessions (filtered by date range)
- License info: expiration date, usage counts vs limits
- Number of admins vs max allowed
- Number of sessions per month vs monthly limit
- Number of assessments per month vs monthly limit

flag: Stats endpoint returns pre-computed values. Cannot verify from code alone whether these are accurately calculated or stale. The organization model computes some stats as properties (e.g., `number_of_sessions_per_month`) which recalculates on every call – could be slow for large orgs.

4.2 Sessions (/dashboard/sessions)

Session List

- **3 tabs:** Active, Finished, Archived
- Each tab has its own table component with independent data fetching
- **Columns:** Worker name, Date, Duration, Status, Limbs, Tags, Actions
- **Filters:** Search, `worker_id`, limbs, status
- **Sorting:** By any column
- **Pagination:** Page-based, configurable `page_size`

Session Statuses

STATUS	MEANING
ONGOING	Currently being recorded on mobile
PAUSED	Recording paused on mobile
FINISHED	Recording completed, data uploaded
FAILED	Recording or processing failed
INTERRUPTED	Recording aborted mid-session

Session Detail Page (</dashboard/sessions/:sessionId/details>)

Data fetched:

- Session metadata: GET </sessions/{sessionId}/>
- Session stats: GET </sessions/{sessionId}/stats/>
- Detailed stats: GET </sessions/{sessionId}/stats/details/>
- TNO stats: GET </sessions/{sessionId}/stats/tno/>
- HR stats: GET </sessions/{sessionId}/stats/hr/>
- Temperature stats: GET </sessions/{sessionId}/stats/temperature/>
- Task stats: GET </sessions/{sessionId}/stats/tasks/>
- Percentile stats: GET </sessions/{sessionId}/stats/percentiles/>
- Session logs: GET </sessions/{sessionId}/logs/>
- Assigned labels: GET </sessions/{sessionId}/assigned-labels/>

What's shown:

- Worker info, duration, date, status, devices used
- Posture charts (elevation angles, bending)
- Speed of movement (angular velocity)
- Wrist distance zones (<30cm, 30-50cm, >50cm)
- HR/HRV data (if available)
- Temperature data per limb
- TNO analysis (static/dynamic movement breakdown)
- Task-by-task breakdown (if events were logged)
- Session logs (notes, media)

Available Actions on a Session

- **Delete:** DELETE </sessions/delete/{sessionId}/>
- **Archive:** POST </sessions/archive-multiple/> (bulk)
- **Export Excel:** GET </sessions/{sessionId}/stats/export/excel/>
- **Export Raw CSV:** GET </sessions/{sessionId}/stats/export/>
- **Export Word Report:** GET </sessions/{sessionId}/export-word/> with options for what to include (metadata, duration, worker, recorder, posture, percentile_stats, temperature_data, risk_assessment, appendix, task_id)
- **Recalculate Stats:** POST </sessions/{sessionId}/stats/recalculate/>

- **Generate MEC Assessment:** POST </sessions/{sessionId}/assesments/mappings/>
- **Generate AI Analysis:** POST </sessions/{sessionId}/generate-gemini-analysis/>
- **Merge Sessions:** POST </sessions/merge/> (combine multiple sessions)
- **Add Log Entry:** POST </sessions/{sessionId}/logs/>
- **Upload ZIP:** POST </sessions/{sessionId}/upload-zip/>

Multi-Session Comparison (</dashboard/sessions/details>)

- Select multiple sessions via checkboxes
- Navigate to comparison view with tabs:
 - **Table of Results** — side-by-side metrics comparison
 - **TNO Results** — compared TNO analysis
 - **Heart Rate Data** — compared HR/HRV
 - **Temperature** — compared temperature data
- Export comparison: GET </sessions/stats/compare/excel/?ids>

Session Archiving

- Bulk archive via POST </sessions/archive-multiple/>
- Per-user archiving settings: GET/PATCH/POST/DELETE </sessions/users/{userId}/archiving-settings/>
- Archived sessions visible in "Archived" tab
- Can be unarchived

4.3 Assessments (</dashboard/assessments>)

Assessment Types

RAMP (Rapid Assessment of Musculoskeletal Posture)

- **Source:** Manual (user fills forms) or Generated (from sensor data + AI)
- **7-step form process:**
 1. **Postures** — 8 posture questions with body diagrams
 2. **Movements and Repetition** — Repetitive movement assessment
 3. **Lifting Work** — Lifting assessment (weights, frequency, posture)
 4. **Pushing and Pulling** — Push/pull force assessment
 5. **Influencing Factors** — Environmental factors (vibration, cold, stress)
 6. **Strenuous Work** — Strenuous activity assessment
 7. **Perceived Physical Discomfort** — Body map for discomfort marking
- **Results page:** Risk score with color coding, detailed breakdown, comments
- **AI Continuation:** `GeneratedRAMPResultsPage` at </continue> sub-route uses Gemini AI to auto-fill/suggest assessment values based on session sensor data
- **Routes:**
 - List: </dashboard/ramp-assessments>
 - Detail: </dashboard/ramp-assessments/:id> (redirects to postures step)
 - Results: </dashboard/ramp-assessments/:id/results>
 - Multi-compare: </dashboard/ramp-assessments/results>

RULA (Rapid Upper Limb Assessment)

- **3-section form:**
 1. **Left Side** — Left arm/hand/wrist analysis
 2. **Right Side** — Right arm/hand/wrist analysis
 3. **Neck, Trunk and Legs** — Head, trunk, leg assessment
- Each section computes individual scores → combined final score
- **Routes:**
 - List: `/dashboard/rula-assessments`
 - Detail: `/dashboard/rula-assessments/:id` (redirects to right side)
 - Results: `/dashboard/rula-assessments/:id/results`
 - Multi-compare: `/dashboard/rula-assessments/results`

REBA (Rapid Entire Body Assessment)

- Similar structure to RULA but assesses whole body
- **Routes:**
 - List: `/dashboard/reba-assessments`
 - Detail: `/dashboard/reba-assessments/:id`
 - Results: `/dashboard/reba-assessments/:id/results`
 - Multi-compare: `/dashboard/reba-assessments/results`

MEC (Metabolic Energy Cost)

- Generated from session sensor data via POST `/sessions/{sessionId}/assesments/mappings/`
- Calculates metabolic cost based on:
 - Arm postures (dominant arm + forearm analysis)
 - Wrist position using geometric calculations
 - 6 work posture models for different arm elevation ranges
 - Frequency analysis of arm movements
- MEC-specific threshold configuration
- **Routes:**
 - Results: `/dashboard/mec-assessments/:id/results`
 - Multi-compare: `/dashboard/mec-assessments/results`

Assessment Statuses

STATUS	MEANING
COMPLETED	Assessment fully filled and results available
IN_PROGRESS	Assessment partially filled (draft)
PROCESSING	Assessment being generated/calculated
FAILED	Assessment generation/calculation failed

Assessment Sources

- **manual** — User manually filled the form
- **generated** — Auto-generated from sensor data (MEC) or AI-assisted (RAMP via Gemini)

Assessment Actions

- Create: POST `/assesments/`
- View/Edit: GET/PATCH `/assesments/{id}/`
- Delete (bulk): DELETE `/assesments/`
- Fetch multiple for comparison: GET `/assesments/multi/?ids`
- Export AVIX format: POST `/assesments/avix-export/`

4.4 Reports & AI Generation

Gemini AI Analysis

- **Trigger:** POST `/sessions/{sessionId}/generate-gemini-analysis/`
- **Input:** Session data + language + optional work label tags
- **What the AI receives (prompt):**
 - Worker profile (age, weight, dominant arm)
 - Measurement duration with extrapolation caveat
 - Posture & movement data with risk zone thresholds
 - Arm elevation analysis (50th/90th percentile limits)
 - Trunk bending analysis (20-45° and >45° thresholds)
 - Wrist angular velocity data
 - HR/HRV stress indicators
- **Risk Zone Thresholds in Prompt:**
 - ARM ELEVATION: Green <25%, Yellow 25-50%, Red >50%
 - TRUNK BENDING 20-45°: Green <12.5%, Yellow 12.5-25%, Red >25%
 - TRUNK BENDING >45°: Green <6%, Yellow 6-12.5%, Red >12.5%
- **Output:** Actionable ergonomic recommendations in the requested language
- **Languages:** Full localized prompts in EN, SV, DE, ES, NL
- **Stored:** In `Session.ai_report` JSONField
- **Retrieve cached:** GET `/sessions/{sessionId}/generate-gemini-analysis/`
- **AI usage tracked:** `AIRequestLog` model records org + timestamp

Report Templates

- Users can create custom report templates defining what to include
- CRUD via `/users/{userId}/report-templates/`
- Templates specify: metadata, duration, worker, recorder, posture, percentile stats, temperature, risk assessment, appendix

Word Report Export

- GET `/sessions/{sessionId}/export-word/?metadata=true&duration=true&...`
- Granular control over sections included
- Returns downloadable Word document (blob)

Excel Export

- Single session: GET `/sessions/{sessionId}/stats/export/excel/`

- Multi-session comparison: GET `/sessions/stats/compare/excel/?ids`

Raw Data Export

- GET `/sessions/{sessionId}/stats/export/` — Downloads raw measurement CSV data

PDF Generation

- Client-side PDF generation via `@react-pdf/renderer`
- Used for assessment results rendering

4.5 Tags / Configuration System

Category & Label System

Categories:

- Hierarchical tree structure (MPTT — Modified Preorder Tree Traversal)
- Types: `WORK_TASK`, `WORK_LABEL`
- Belong to an organization
- Can be nested (parent/child relationships)

Labels:

- Belong to a category
- Can be public or private (`is_private`)
- Owner tracked
- Can be favorited by users (`favorited_by` M2M)
- Optional repeat interval

Events:

- Link labels to specific time ranges in a session
- Each event has: `start_time`, `end_time`, `labels` (M2M), `created_by`
- Used for task-by-task analysis

CRUD Routes:

- Categories: GET/POST `/categories/organizations/{orgId}/category-with-labels/`
- Labels: GET/POST `/categories/{categoryId}/labels/`
- Events: GET/POST `/categories/labels/events/sessions/{sessionId}/`

Work Cycles (Factory Hierarchy)

Structure: Factory → Line → Workstation → TaskModel → Task

LEVEL	DESCRIPTION	UNIQUE PER
Factory	Physical factory/site	Organization
Line	Production line within factory	Factory
Workstation	Work station within line	Line
TaskModel	Task template at a workstation	Workstation (via Operator in web)
Task	Specific task instance	TaskModel

Each Task has: name, description, duration, created_at

Features:

- Full CRUD on each level
- Full hierarchy view: GET `/workcycles/factories/full-hierarchy/`
- Import from AVIX file: POST `/workcycles/import/`
- Import from Excel: POST `/workcycles/import-excel/`
- Work cycles linked to sessions via `Session.workcycle_task` FK

4.6 User & Organization Management

User Management (`/dashboard/users`)

- List users with filters: role, active status, search, ordering
- Add new user to organization
- Update user details (name, role, email, personal number)
- Deactivate user (soft disable, not delete)
- Delete user (hard delete — admin only)
- Bulk operations

Work Profiles (`/dashboard/work-profiles`)

- List worker profiles
- Manage extended profile info: weight, heart rate, gender, DOB, dominant arm
- Assign consultant (ergonomist) to worker

Organization Settings

- View/edit org details (name, country, industry, etc.)
- Upload/change organization logo
- View license info and limits
- View AI request usage per month: GET `/organizations/{orgId}/ai-requests-per-month/`

4.7 Charts & Analytics

Chart Library

- **Nivo 0.83.0** — ResponsiveLine, ResponsiveBar, ResponsivePie, ResponsiveDonut

Charts Available

Session Detail Charts:

- **Posture Line Chart** (PaginatedLineChart) — Time-series elevation angle data per limb with zoom, pagination, downsampling, and log event markers
- **Angular Velocity Line Chart** — Speed of movement over time
- **Donut Charts** — Distribution of time in risk zones (green/yellow/red)
- **Trunk Bending Charts** — Forward and side bending angles over time
- **Wrist Distance Zones** — Pie chart: <30cm, 30-50cm, >50cm distribution
- **Heart Rate Chart** — BPM over time with HRV metrics
- **Temperature Chart** — Per-limb temperature trends
- **TNO Charts** — Static/dynamic movement breakdown per angle range

Dashboard Charts:

- UsersStatistics cards — Numeric metrics
- SessionsStatistics cards — Session counts with date filtering
- LicenseStatistics cards — Usage vs limits

Multi-Session Comparison:

- Side-by-side bar charts for risk scores
- Comparison tables

Data Transformation

- Raw API data consumed directly
- Large datasets downsampled via `downsample()` utility to prevent overplotting
- Custom tooltips show X/Y values + associated log events
- Color coding from theme system

4.8 Data Export

Export Options

FORMAT	FROM	ENDPOINT
Excel (.xlsx)	Single session stats	<code>/sessions/{id}/stats/export/excel/</code>
Excel (.xlsx)	Multi-session comparison	<code>/sessions/stats/compare/excel/?ids</code>
Word (.docx)	Session report	<code>/sessions/{id}/export-word/</code>
CSV	Raw measurement data	<code>/sessions/{id}/stats/export/</code>
AVIX	Assessments	<code>/assessments/avix-export/</code>
PDF	Assessment results	Client-side via <code>@react-pdf/renderer</code>

Word Export Options

Granular control — each section toggleable:

- Metadata, Duration, Worker info, Recorder info

- Posture data, Percentile stats
- Temperature data, Risk assessment
- Appendix, Task-specific data

4.9 Notifications

Notification System

- Backend creates notifications for events: MEC_ASSESSMENT, SESSION, RAMP_ASSESSMENT
- Pushed to Firestore on creation (signal handler)
- Web fetches via REST: GET `/notifications/?page&page_size`
- Mark as read: POST `/notifications/mark-as-read/`
- Delete: DELETE `/notifications/{id}/`
- Each notification has: title, message, type, created_at, read_at, foreign_key (link to related object)

4.10 Support Tickets

- Create support tickets: POST `/tickets/`
- Types: bug, feature, suggestion, help
- Help subtypes: account, billing, other
- Fields: subject, message, attachments
- List tickets: GET `/tickets/`

Admin Panel — Complete Feature Map

Access: WERGONIC_ADMIN role only

Routes

PATH	PAGE	PURPOSE
/login	LoginPage	Admin login
/organizations	OrganizationsPage	List/manage all organizations
/users	UsersPage	List/manage all users across orgs
/devices	DevicesPage	List/manage all devices
/firmwares	FirmwarePage	Manage firmware versions

Organization Management

- List all organizations with filters: country, active status, archived status, license expiration range
- Search and sort
- Create new organization (with FormData for logo upload)
- Edit organization details
- Archive/unarchive organizations (bulk): POST /organizations/change-to-archive/
- View all org names: GET /organizations/names

User Management

- List all users across all organizations
- Filter by: organization, role, search term
- Add users to specific organizations
- Deactivate users from organizations
- View user-organization relationships

Device Management

- List all registered devices: GET /devices/?search&page&page_size
- View device details: serial, firmware, organization, worker assignment

Firmware Management

- List firmware versions: GET /devices/firmwares/?search&page&page_size&is_current
- Create new firmware entry: POST /devices/firmwares/
- Update firmware details: PATCH /devices/firmwares/{id}/
- Mark firmware as current (is_current flag)
- Firmware fields: description, mobile_version, release_date, file_size, firmware_version, firmware_url

Notifications & Feedback Systems

In-App Notifications (Mobile)

- Firebase push notifications via Firestore
- Notification written to Firestore on backend event (post_save signal)
- Path: `organizations/{orgId}/users/{userId}/notifications/{notificationId}/`
- Notification types: MEC_ASSESSMENT, SESSION, RAMP_ASSESSMENT
- Each contains: title + associated object ID (foreign_key)

Web Notifications

- REST-based polling via GET `/notifications/`
- Paginated list with read/unread status
- Mark as read functionality
- No WebSocket/real-time push on web (Channels installed but not implemented)

Snackbar Notifications (Web)

- notistack library for toast-style notifications
- Success/error/info messages for CRUD operations
- Auth error messages (e.g., worker can't access)

Error Messages

- API errors converted to user-friendly messages
- Mobile: `ApiException` with localized display messages (401 → "Session expired", 403 → "No permission", etc.)
- Web: Axios interceptor returns Promise.reject, component-level error handling via React Query error states

Email Notifications

- Session processing completion email (with elapsed time)
- Invitation emails (7-day expiry)
- Password reset emails
- Welcome emails after invitation acceptance
- Magic link emails for passwordless login

Known Issues & Gaps (from code analysis)

Unfinished Features

Vibration Detection Display (Mobile)

- Backend processing exists
- Mobile has `isVibrationDetectionEnabled` setting and improved detection algorithm
- UI panels exist but have TODOs: "Implement time display + limb toggle" at `vibration_detection_panel.dart:65,85`

Firmware Update Flow (Mobile)

- DFU library integrated (nordic_dfu)
- Firmware screen exists (`/firmware`)
- Update dialog present but TODO: "Start update flow incomplete" at `firmware_update_available_dialog.dart:25`

Sensor Pairing Actions (Mobile)

- `sensor_vibration_list_item.dart:122,137,149` — Unpair/Pair/Disconnect button handlers not implemented (TODO comments)

Work Cycle Auto-Advance (Mobile)

- Timer logic exists via `timeBasedWorkCycles` setting
- Default frequency 180 minutes
- UI for auto-advance not fully wired

WebSocket Support (Backend)

- Django Channels 4.1.0 installed
- ASGI configuration exists
- No WebSocket consumers or routing implemented
- Real-time features rely on Firestore instead

Placeholder/Dummy Elements

Testing Pages (Web)

- `TestingNewComponentsPage` — Empty dev testing page (hidden behind feature flag)
- `IconsPage` — Icon showcase (hidden behind feature flag)
- Both only accessible in development mode

Legacy API Routes (Web)

- `legacyApiRoutes.ts` — 17KB file of apparently unused routes from old architecture
- Candidate for cleanup

Dead Code

Commented Proxy Models (Backend)

- `workerModel.py`, `ergonomistModel.py`, `orgAdminModel.py` — All commented out proxy model files
- Were supposed to be role-specific model extensions but abandoned

Test Runner (Backend)

- `test_runner.py` — Commented out test runner configuration

Unused Imports (Mobile)

- Some unused imports not cleaned up across Flutter codebase

Statistics & Metrics Concerns

Organization Stats Calculation

- `number_of_sessions_per_month` and `number_of_assessments_per_month` are computed as model properties
- Recalculated on every API call — no caching
- Could be slow for organizations with many sessions

Session Processing Pipeline

- Uses Python `Thread` for background processing (not a proper task queue)
- No retry logic or dead-letter handling
- If processing fails, session stays in unprocessed state with no automatic recovery

Security Concerns

ALLOWED_HOSTS

- `settings.py:24` — `ALLOWED_HOSTS = ["*"]` (accepts requests from any host)

Password in Email

- `organizationView.py:347` — Password emailed during user creation

Magic Link Exposure

- URL-encoded magic link passed as query parameter — could be logged by proxies

Missing Rate Limiting

- Only `custom_post: 10/minute` throttle exists
- No rate limiting on login or password reset endpoints

Missing Translations

- i18n configured for 5 languages but completeness of translations not verified
- Some assessment forms may have untranslated labels

TODO/FIXME/HACK Items

Backend

LOCATION	COMMENT
<code>Session_post_save</code> signal	Spawns Thread without task queue — no retry, no monitoring
<code>firebase_auth/authentication.py:49</code>	Bare <code>except:</code> clause with no specificity
<code>user_sessions/models.py:262-275</code>	Commented multiprocessing approach for TNO
Cron config <code>settings.py:295-296</code>	Hardcoded <code>/usr/local/bin/python3</code> path

Mobile

LOCATION	COMMENT
<code>sensor_vibration_list_item.dart:122,137,149</code>	Unpair/Pair/Disconnect not implemented
<code>firmware_update_available_dialog.dart:25</code>	Start update flow incomplete
<code>work_label_model.dart:25</code>	Replace name with ID mapping
<code>switch_organization_section.dart:34</code>	Review code quality
<code>sensors_settings_section.dart:208</code>	Restore deferred feature
<code>feedback_display_panel.dart:123</code>	Move logic elsewhere
<code>vibration_detection_panel.dart:65,85</code>	Implement time display + limb toggle
<code>session.remote.datasource.dart:112,146</code>	Review logic + error handling

Web

LOCATION	COMMENT
<code>apiRoutes.ts:1</code> (client-panel)	<code>// todo delete all comments</code> — hundreds of cleanup comments
<code>TestingNewComponentsPage:12</code>	<code>console.log(allDataIsValid)</code> left in
<code>auth.ts:61</code> (admin)	<code>console.error</code> instead of proper logging

Inconsistencies

Naming

- `assesment_category` field name (typo — should be "assessment") throughout backend
- `switch0ranization` route name (typo — should be "Organization") in mobile
- Mixed snake_case and camelCase in JSON fields between backend and frontend
- `VITE_API_BASE_URL` env variable (typo — "URI" instead of "URL")

Data Consistency

- Firestore data not always synced with PostgreSQL on session updates
- CSV file references in Session can become orphaned if Firebase Storage file is deleted
- No cascade delete validation

Complete Route Map

Web Client Panel Routes

PATH	PAGE	AUTH
/auth/login	LoginPage	Public
/auth/login/email	EmailLoginPage	Public
/auth/signup	AccountCreationPage	Public
/auth/password-reset	InitialPasswordResetPage	Public
/auth/password-reset/new	PasswordResetPage	Public
/auth/process-login	ProcessEmailLinkLogin	Public
/auth/mobile-redirect	Mobile redirect	Public
/dashboard	HomePage	Auth Required
/dashboard/users	UsersPage	Auth Required
/dashboard/work-profiles	WorkProfilesPage	Auth Required
/dashboard/workcycles	WorkCyclesPage	Auth Required
/dashboard/sessions	SessionsPage (3 tabs)	Auth Required
/dashboard/sessions/:id/details	SessionDetailsPage	Auth Required
/dashboard/sessions/details	MultiSessionDetailsPage	Auth Required
/dashboard/assessments	AssessmentsPage	Auth Required
/dashboard/category-and-labels	CategoryAndLabelsPage	Auth Required
/dashboard/ramp-assessments	RAMPAssessmentsPage	Auth Required
/dashboard/ramp-assessments/:id	RAMPDetailPage (7 sub-routes)	Auth Required
/dashboard/ramp-assessments/:id/results	RAMPResultsPage	Auth Required
/dashboard/ramp-assessments/results	MultiRAMPResultsPage	Auth Required
/dashboard/rula-assessments	RULAAssessmentsPage	Auth Required
/dashboard/rula-assessments/:id	RULADetailPage (3 sub-routes)	Auth Required
/dashboard/rula-assessments/:id/results	RULAResultsPage	Auth Required
/dashboard/rula-assessments/results	MultiRULAResultsPage	Auth Required
/dashboard/reba-assessments	REBAAssessmentsPage	Auth Required
/dashboard/reba-assessments/:id	REBADetailPage	Auth Required

PATH	PAGE	AUTH
/dashboard/reba-assessments/:id/results	REBAResultsPage	Auth Required
/dashboard/reba-assessments/results	MultiREBAResultsPage	Auth Required
/dashboard/mec-assessments/:id/results	MECResultsPage	Auth Required
/dashboard/mec-assessments/results	MultiMECResultsPage	Auth Required
/dashboard/test	TestingNewComponentsPage	Dev Only
/dashboard/icons	IconsPage	Dev Only
/privacy-policy	PrivacyPolicyPage	Public
/privacy-and-data-deletion-policy	PrivacyAndDataDeletionPolicyPage	Public
/invitations/:id	InvitationPage	Public
/error	ErrorPage	Public

Admin Panel Routes

PATH	PAGE	AUTH
/login	LoginPage	Public
/organizations	OrganizationsPage	WERGONIC_ADMIN
/users	UsersPage	WERGONIC_ADMIN
/devices	DevicesPage	WERGONIC_ADMIN
/firmwares	FirmwarePage	WERGONIC_ADMIN
/error	ErrorPage	Public

Mobile App Routes

PATH	SCREEN	PURPOSE
/	SplashScreen	Init/auth check
/login	LoginPage	OAuth login
/login_with_email	LoginWithEmailPage	Email login
/reset_password	ResetPasswordScreen	Password reset request
/check_reset_password	CheckLinkResetPasswordScreen	Email link validation
/link_expired_reset_password	LinkExpiredResetPasswordScreen	Expired link
/create_reset_password	CreateNewPasswordScreen	New password creation
/welcome	WelcomeScreen	Post-login intro
/settings	SettingScreen	All settings
/settings/languages	LanguagesSettingsPage	Language picker
/settings/value-picker	ValuePickerScreen	Generic value picker
/settings/switch_oranization	SwitchOrganizationScreen	Org switcher
/limbsFeedbackOrder	LimbsFeedbackPriorityScreen	Feedback priority
/work_session_config	WorkSessionConfig	Session setup
/work_session_config/work-cycle-screen	WorkCycleScreen	Work cycle selection
/work_session_config/work-cycle-task-screen	WorkCycleTasksListScreen	Work cycle tasks
/work_session_config/pairing	PairingScreen	BLE sensor pairing
/firmware	FirmwareScreen	Firmware management
/recalibration	LimbsRecalibrationPage	Per-limb recalibration
/sensor-calibration	LimbsCalibrationPage	Full calibration
/work_session	WorkSessionRecordingScreen	Session recording
/work_session/WorkTasks	WorkTasksScreen	Task selection
/sessions_listing	WorkSessionListPage	All sessions
/session_summary	SessionSummaryPage	Post-session summary

PATH	SCREEN	PURPOSE
/session_history	SessionHistoryPage	Past sessions + sync
/session_history/add_work_label	AddWorkLabelToSessionHistoryScreen	Add task/label
/report	ReportScreen	Session analysis
/tnoReport	TnoReportPage	TNO risk report
/reportProblem	ReportProblemScreen	Bug reporting
/liteMode	LiteModeCalibrationPage	Phone-based setup

Wergonic — internal documentation. Generated from codebase analysis of the `dev` branch (backend, web apps, mobile) on 17 March 2026.